

```

1 //#####
2 // FILE:   FinalProjectStarter_main.c
3 //
4 // TITLE:   Final Project Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "F2837xD_SWPrioritizedIsrLevels.h"
16 #include "driverlib.h"
17 #include "device.h"
18 #include "F28379dSerial.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22 #include "xy.h"
23 #include "MatrixMath.h"
24 #include "SE423Lib.h"
25 #include "OptiTrack.h"
26
27 #define PI          3.1415926535897932384626433832795
28 #define TWOPI       6.283185307179586476925286766559
29 #define HALFPI      1.5707963267948966192313216916398
30 // The Launchpad's CPU Frequency set to 200 you should not change this value
31 #define LAUNCHPAD_CPU_FREQUENCY 200
32
33 // ----- code for CAN start here -----
34 #include "F28379dCAN.h"
35 // #define TX_MSG_DATA_LENGTH      4
36 // #define TX_MSG_OBJ_ID           0 //transmit
37
38 #define RX_MSG_DATA_LENGTH      8
39 #define RX_MSG_OBJ_ID_1         1 //measurement from sensor 1
40 #define RX_MSG_OBJ_ID_2         2 //measurement from sensor 2
41 #define RX_MSG_OBJ_ID_3         3 //quality from sensor 1
42 #define RX_MSG_OBJ_ID_4         4 //quality from sensor 2
43 // ----- code for CAN end here -----
44
45 // Interrupt Service Routines predefinition
46 __interrupt void cpu_timer0_isr(void);
47 __interrupt void cpu_timer1_isr(void);
48 __interrupt void cpu_timer2_isr(void);
49 __interrupt void SWI1_HighestPriority(void);
50 __interrupt void SWI2_MiddlePriority(void);
51 __interrupt void SWI3_LowestPriority(void);
52 __interrupt void ADCC_ISR(void);
53 __interrupt void SPIB_isr(void);
54 // ----- code for CAN start here -----
55 __interrupt void can_isr(void);
56 // ----- code for CAN end here -----
57
58 void setF28027EPWM1A(float controleffort);
59 int16_t EPwm1A_F28027 = 1500;
60 void setF28027EPWM2A(float controleffort);
61 int16_t EPwm2A_F28027 = 1500;
62
63 // ----- code for CAN start here -----
64 // volatile uint32_t txMsgCount = 0;
65 // extern uint16_t txMsgData[4];
66
67 volatile uint32_t rxMsgCount_1 = 0;
68 volatile uint32_t rxMsgCount_2 = 0;
69 extern uint16_t rxMsgData[8];
70
71 uint32_t dis_raw_1[2];
72 uint32_t dis_raw_2[2];
73 uint32_t dis_1 = 0;
74 uint32_t dis_2 = 0;
75
76 uint32_t quality_raw_1[4];
77 uint32_t quality_raw_2[4];
78 float quality_1 = 0.0;
79 float quality_2 = 0.0;
80
81 uint32_t lightlevel_raw_1[4];
82 uint32_t lightlevel_raw_2[4];
83 float lightlevel_1 = 0.0;
84 float lightlevel_2 = 0.0;
85
86 uint32_t measure_status_1 = 0;
87 uint32_t measure_status_2 = 0;
88
89 volatile uint32_t errorFlag = 0;
90 // ----- code for CAN end here -----
91
92 uint32_t numTimer0calls = 0;
93 uint16_t UARTPrint = 0;
94
95 float printLV1 = 0;
96 float printLV2 = 0;
97
98 float printLinux1 = 0;
99 float printLinux2 = 0;
100
101 uint16_t LADARi = 0;
102 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
103 uint16_t G_len = 11; //length of command
104 xy_ladar_pts[228]; //xy data
105 float LADARrightfront = 0;
106 float LADARfront = 0;
107 float LADARtemp_x = 0;
108 float LADARtemp_y = 0;
109 extern datapts_ladar_data[228];
110
111 extern uint16_t newLinuxCommands;
112 extern float LinuxCommands[CMDNUM_FROM_FLOATS];

```

```

113
114 extern uint16_t NewLVData;
115 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
116 extern LVSendFloats_t DataToLabView;
117 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
118 extern uint16_t LADARpingpong;
119 extern uint16_t NewCAMDataThreshold1; // Flag new data
120 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
121 extern uint16_t NewCAMDataThreshold2; // Flag new data
122 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
123
124 extern uint16_t NewCAMDataAprilTag1; // Flag new data
125 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
126
127 float tagid = 0;
128 float tagx = 0;
129 float tagy = 0;
130 float tagz = 0;
131 float tagthetax = 0;
132 float tagthetay = 0;
133 float tagthetaz = 0;
134 int32_t numtag = 0;
135 int32_t AprilTagState = 10;
136 float KpAprilTag = -1.0;
137 float AprilTagSetPoint = 0;
138 int32_t AprilTagCount = 0;
139
140 float MaxAreaThreshold1 = 0;
141 float MaxColThreshold1 = 0;
142 float MaxRowThreshold1 = 0;
143 float NextLargestAreaThreshold1 = 0;
144 float NextLargestColThreshold1 = 0;
145 float NextLargestRowThreshold1 = 0;
146 float NextNextLargestAreaThreshold1 = 0;
147 float NextNextLargestColThreshold1 = 0;
148 float NextNextLargestRowThreshold1 = 0;
149
150 float MaxAreaThreshold2 = 0;
151 float MaxColThreshold2 = 0;
152 float MaxRowThreshold2 = 0;
153 float NextLargestAreaThreshold2 = 0;
154 float NextLargestColThreshold2 = 0;
155 float NextLargestRowThreshold2 = 0;
156 float NextNextLargestAreaThreshold2 = 0;
157 float NextNextLargestColThreshold2 = 0;
158 float NextNextLargestRowThreshold2 = 0;
159
160 uint32_t numThres1 = 0;
161 uint32_t numThres2 = 0;
162
163 pose ROBOTps = {0,0,0}; //robot position
164 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
165 float LADARxoffset = 0;
166 float LADARyoffset = 0;
167
168 uint32_t timecount = 0;
169 int16_t RobotState = 1;
170 int16_t checkfronttally = 0;
171 int32_t WallFollowtime = 0;
172
173 #define NUMWAYPOINTS 8
174 uint16_t statePos = 0;
175 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
176 uint16_t i = 0; //for loop
177
178 uint16_t right_wall_follow_state = 2; // right follow
179 float Kp_front_wall = -2.0;
180 float front_turn_velocity = 0.2;
181 float left_turn_Stop_threshold = 3.5;
182 float Kp_right_wal = -4.0;
183 float ref_right_wall = 1.1;
184 float foward_velocity = 1.0;
185 float left_turn_Start_threshold = 1.3;
186 float turn_saturation = 2.5;
187
188 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
189
190 //more kalman vars
191 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
192 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
193 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
194 float z[3][1]; // state measurement
195 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
196 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
197 #define ProcUncert 0.0001
198 #define CovScalar 10
199 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
200 {0,ProcUncert,ProcUncert/CovScalar},
201 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
202 #define MeasUncert 1
203 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
204 {0,MeasUncert,MeasUncert/CovScalar},
205 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of Optitrack Motion Capture measurement)
206 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
207 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
208 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
209 float temp_3x3[3][3]; // intermediate storage
210 float temp_3x1[3][1]; // intermediate storage
211 float ytilde[3][1]; // difference between predictions
212
213 // Optitrack Variables
214 int32_t OptiNumUpdatesProcessed = 0;
215 pose OPTITRACKps;
216 extern uint16_t new_optitrack;
217 extern float Optitrackdata[OPTITRACKDATASIZE];
218 int16_t newOPTITRACKpose=0;
219
220 int16_t adcc2result = 0;
221 int16_t adcc3result = 0;
222 int16_t adcc4result = 0;
223 int16_t adcc5result = 0;
224 float adcc2Volt = 0.0;
225 float adcc3Volt = 0.0;
226 float adcc4Volt = 0.0;

```

```

227 float adcC5Volt = 0.0;
228 int32_t numADCCalls = 0;
229 float LeftWheel = 0;
230 float RightWheel = 0;
231 float LeftWheel_1 = 0;
232 float RightWheel_1 = 0;
233 float LeftVel = 0;
234 float RightVel = 0;
235 float uLeft = 0;
236 float uRight = 0;
237 float HandValue = 0;
238 float vref = 0;
239 float turn = 0;
240 float gyro9250_drift = 0;
241 float gyro9250_angle = 0;
242 float old_gyro9250 = 0;
243 float gyroLPR510_angle = 0;
244 float gyroLPR510_offset = 0;
245 float gyroLPR510_drift = 0;
246 float old_gyroLPR510 = 0;
247 float gyro9250_radians = 0;
248 float gyroLPR510_radians = 0;
249
250 int16_t readdata[25];
251 int16_t IMU_data[9];
252
253 float accelx = 0;
254 float accely = 0;
255 float accelz = 0;
256 float gyrox = 0;
257 float gyroy = 0;
258 float gyroz = 0;
259
260 // Needed global Variables
261 float accelx_offset = 0;
262 float accely_offset = 0;
263 float accelz_offset = 0;
264 float gyrox_offset = 0;
265 float gyroy_offset = 0;
266 float gyroz_offset = 0;
267 int16_t calibration_state = 0;
268 int32_t calibration_count = 0;
269 int16_t doneCal = 0;
270
271 #define MPU9250 1
272 #define DAN28027 2
273 int16_t CurrentChip = MPU9250;
274 int16_t DAN28027Garbage = 0;
275 int16_t dan28027adc1 = 0;
276 int16_t dan28027adc2 = 0;
277 uint16_t MPU9250ignoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
278
279 float RCangle;
280
281 void main(void)
282 {
283     // PLL, WatchDog, enable Peripheral Clocks
284     // This example function is found in the F2837xD_SysCtrl.c file.
285     InitSysCtrl();
286
287     InitGpio();
288
289     InitSE423DefaultGPIO();
290
291     //Scope for Timing
292     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
293     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
294     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
295
296     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
297     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
298     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
299
300     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
301     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
302     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
303
304     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
305     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
306     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
307
308     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
309     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
310     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
311
312     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
313     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
314     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
315
316 // ----- code for CAN start here -----
317 //GPIO17 - CANRXB
318 GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
319 GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
320
321 //GPIO12 - CANTXB
322 GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
323 GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
324 // ----- code for CAN end here -----
325
326
327
328 // ----- code for CAN start here -----
329 // Initialize the CAN controller
330 InitCANB();
331
332 // Set up the CAN bus bit rate to 1000 kbps
333 setCANBitRate(200000000, 1000000);
334
335 // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
336 CanbRegs.CAN_CTL.bit.IE0= 1;
337 CanbRegs.CAN_CTL.bit.EIE= 1;
338 // ----- code for CAN end here -----
339
340 // Clear all interrupts and initialize PIE vector table:

```

```

341 // Disable CPU interrupts
342 DINT;
343
344 // Initialize the PIE control registers to their default state.
345 // The default state is all PIE interrupts disabled and flags
346 // are cleared.
347 // This function is found in the F2837xD_PieCtrl.c file.
348 InitPieCtrl();
349
350 // Disable CPU interrupts and clear all CPU interrupt flags:
351 IER = 0x0000;
352 IFR = 0x0000;
353
354 // Initialize the PIE vector table with pointers to the shell Interrupt
355 // Service Routines (ISR).
356 // This will populate the entire table, even if the interrupt
357 // is not used in this example. This is useful for debug purposes.
358 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
359 // This function is found in F2837xD_PieVect.c.
360 InitPieVectTable();
361
362 // Interrupts that are used in this example are re-mapped to
363 // ISR functions found within this project
364 EALLOW; // This is needed to write to EALLOW protected registers
365 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
366 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
367 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
368 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
369 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
370 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
371 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
372 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
373 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
374 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
375 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
376 PieVectTable.ADCCL_INT = &ADCC_ISR;
377 PieVectTable.SPIB_RX_INT = &SPIB_isr;
378 PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
379 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
380 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
381 // ----- code for CAN start here -----
382 PieVectTable.CANB0_INT = &can_isr;
383 // ----- code for CAN end here -----
384 EDIS; // This is needed to disable write to EALLOW protected registers
385
386 // Initialize the CpuTimers Device Peripheral. This function can be
387 // found in F2837xD_CpuTimers.c
388 InitCpuTimers();
389
390 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
391 // 200MHz CPU Freq, Period (in uSeconds)
392 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
393 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms. Do not Change.
394 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
395
396 // Enable CpuTimer Interrupt bit TIE
397 CpuTimer0Regs.TCR.all = 0x4000;
398 CpuTimer1Regs.TCR.all = 0x4000;
399 CpuTimer2Regs.TCR.all = 0x4000;
400
401 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
402
403 init_serialSCIB(&SerialB,19200);
404 init_serialSCIC(&SerialC,19200);
405
406 for (LADARi = 0; LADARi < 228; LADARi++) {
407     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
408 }
409
410 init_eQEPs();
411 init_EPWM1and2();
412 init_RCServoPWM_3AB_5AB_6A();
413 init_ADCsAndDACs();
414 setupSpib();
415
416 EALLOW;
417 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
418 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
419 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
420 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
421 EPwm4Regs.TBCTR = 0x0; // Clear counter
422 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
423 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
424 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
425 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
426 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
427 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
428 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
429 EDIS;
430
431 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
432 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
433 // to CPU-Timer 2: int 12 is for the SWI.
434 IER |= M_INT1;
435 IER |= M_INT6;
436 IER |= M_INT8; // SCIC SCID
437 IER |= M_INT9; // SCIA
438 IER |= M_INT12;
439 IER |= M_INT13;
440 IER |= M_INT14;
441
442 // Enable TINT0 in the PIE: Group 1 interrupt 7
443 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
444 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
445 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
446 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
447 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
448 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
449 // ----- code for CAN start here -----
450 // Enable CANB in the PIE: Group 9 interrupt 7
451 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
452 // ----- code for CAN end here -----
453
454 // ----- code for CAN start here -----

```

```

455 // Enable the CAN interrupt signal
456 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
457 // ----- code for CAN end here -----
458
459
460 robotdest[0].x = -4;    robotdest[0].y = 10;
461 robotdest[1].x = -4;    robotdest[1].y = 2;
462 //middle of bottom
463 robotdest[2].x = 0;     robotdest[2].y = 2;
464 //outside the course
465 robotdest[3].x = 0;     robotdest[3].y = -3;
466 //back to middle
467 robotdest[4].x = 0;     robotdest[4].y = 2;
468 robotdest[5].x = 4;     robotdest[5].y = 2;
469 robotdest[6].x = 4;     robotdest[6].y = 10;
470 robotdest[7].x = 0;     robotdest[7].y = 9;
471
472 // ROBOTps will be updated by Optitrack during gyro calibration
473 // TODO: specify the starting position of the robot
474 ROBOTps.x = 0;          //the estimate in array form (useful for matrix operations)
475 ROBOTps.y = 0;
476 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
477 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
478 x_pred[1][0] = ROBOTps.y;
479 x_pred[2][0] = ROBOTps.theta;
480
481 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
482 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
483 SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
484 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
485 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
486 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
487
488 init_serialSCIA(&SerialA,115200);
489 init_serialSCID(&SerialD,2083332);
490
491 // Enable global Interrupts and higher priority real-time debug events
492 EINT; // Enable Global interrupt INTM
493 ERTM; // Enable Global realtime interrupt DBGM
494 // ----- code for CAN start here -----
495 // Measured Distance from 1
496 // Initialize the receive message object 1 used for receiving CAN messages.
497 // Message Object Parameters:
498 //     Message Object ID Number: 1
499 //     Message Identifier: 0x060b0101
500 //     Message Frame: Standard
501 //     Message Type: Receive
502 //     Message ID Mask: 0x0
503 //     Message Object Flags: Receive Interrupt
504 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
505 //     for a Receive mailbox)
506 //
507 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
508                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
509                       RX_MSG_DATA_LENGTH);
510
511 // Measured Distance from 2
512 // Initialize the receive message object 2 used for receiving CAN messages.
513 // Message Object Parameters:
514 //     Message Object ID Number: 2
515 //     Message Identifier: 0x060b0102
516 //     Message Frame: Standard
517 //     Message Type: Receive
518 //     Message ID Mask: 0x0
519 //     Message Object Flags: Receive Interrupt
520 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
521 //     for a Receive mailbox)
522 //
523 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
524                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
525                       RX_MSG_DATA_LENGTH);
526
527 // Measurement Quality from 1
528 // Initialize the receive message object 2 used for receiving CAN messages.
529 // Message Object Parameters:
530 //     Message Object ID Number: 3
531 //     Message Identifier: 0x060b0201
532 //     Message Frame: Standard
533 //     Message Type: Receive
534 //     Message ID Mask: 0x0
535 //     Message Object Flags: Receive Interrupt
536 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
537 //     for a Receive mailbox)
538 //
539 //
540 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
541                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
542                       RX_MSG_DATA_LENGTH);
543
544 // Measurement Quality from 2
545 // Initialize the receive message object 2 used for receiving CAN messages.
546 // Message Object Parameters:
547 //     Message Object ID Number: 4
548 //     Message Identifier: 0x060b0202
549 //     Message Frame: Standard
550 //     Message Type: Receive
551 //     Message ID Mask: 0x0
552 //     Message Object Flags: Receive Interrupt
553 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
554 //     for a Receive mailbox)
555 //
556 //
557 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
558                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
559                       RX_MSG_DATA_LENGTH);
560
561 //
562 // Start CAN module operations
563 //
564 CanbRegs.CAN_CTL.bit.Init = 0;
565 CanbRegs.CAN_CTL.bit.CCE = 0;
566
567
568

```

```

569 // ----- code for CAN end here -----
570 char S_command[19] = "S1152000124000\n"; // this change the baud rate to 115200
571 uint16_t S_len = 19;
572 serial_sendSCIC(&SerialC, S_command, S_len);
573
574
575 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
576 init_serialSCIC(&SerialC, 115200);
577
578
579 // LED1 is GPIO22
580 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
581 // LED2 is GPIO94
582 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
583 // LED3 is GPIO95
584 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
585 // LED4 is GPIO97
586 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
587 // LED5 is GPIO111
588 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
589
590
591 // IDLE loop. Just sit and loop forever (optional):
592 while(1)
593 {
594     if (UARTPrint == 1) {
595
596 /*
597         if (readbuttons() == 0) {
598             UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
599             UART_printfLine(1, "x:%.2f y:%.2f a:%.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
600             UART_printfLine(2, "F%.4f R%.4f", LADARfront, LADARrightfront);
601         } else if (readbuttons() == 1) {
602             UART_printfLine(1, "01A:%.0fC:%.0fR:%.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
603             UART_printfLine(2, "P1A:%.0fC:%.0fR:%.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
604             //UART_printfLine(1, "LV1:%.3f LV2:%.3f", printLV1, printLV2);
605             //UART_printfLine(2, "Ln1:%.3f Ln2:%.3f", printLinux1, printLinux2);
606         } else if (readbuttons() == 2) {
607             UART_printfLine(1, "02A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
608             UART_printfLine(2, "P2A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
609             // UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
610             // UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
611         } else if (readbuttons() == 4) {
612             UART_printfLine(1, "03A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
613             UART_printfLine(2, "P3A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
614             // UART_printfLine(1, "L:%.3f R:%.3f", LeftVel, RightVel);
615             // UART_printfLine(2, "uL:%.2f uR:%.2f", uLeft, uRight);
616         } else if (readbuttons() == 8) {
617             UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
618             UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
619         } else if (readbuttons() == 3) {
620             UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
621             UART_printfLine(2, "MPU:%.2f LPR:%.2f", gyro9250_radians, gyroLPR510_radians);
622         } else if (readbuttons() == 5) {
623             UART_printfLine(1, "0x:%.2f 0y:%.2f 0a:%.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
624             UART_printfLine(2, "State:%d : %d", RobotState, statePos);
625         } else if (readbuttons() == 6) {
626             UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
627             UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
628         } else if (readbuttons() == 7) {
629             UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
630             UART_printfLine(2, "%.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
631         }
632     }
633     UARTPrint = 0;
634 }
635 }
636 }
637
638 __interrupt void SPIB_isr(void){
639
640     uint16_t i;
641     GpioDataRegs.GPASET.bit.GPIO32 = 1;
642
643     GpioDataRegs.GPCSET.bit.GPIO66 = 1; // Pull CS high as done R/W
644     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
645
646     if (CurrentChip == MPU9250) {
647
648         for (i=0; i<8; i++) {
649             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
650         }
651
652         PostSWI1(); // Manually cause the interrupt for the SWI1
653
654     } else if (CurrentChip == DAN28027) {
655
656         DAN28027Garbage = SpibRegs.SPIRXBUF;
657         dan28027adc1 = SpibRegs.SPIRXBUF;
658         dan28027adc2 = SpibRegs.SPIRXBUF;
659         CurrentChip = MPU9250;
660         SpibRegs.SPIFFCT.bit.TXDLY = 0;
661     }
662
663     SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag
664     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
665     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
666     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
667 }
668
669
670 //adc1 pie interrupt
671 __interrupt void ADCC_ISR (void)
672 {
673     GpioDataRegs.GPASET.bit.GPIO11 = 1;
674     adcc2result = AdccResultRegs.ADCRESULT0;
675     adcc3result = AdccResultRegs.ADCRESULT1;
676     adcc4result = AdccResultRegs.ADCRESULT2;
677     adcc5result = AdccResultRegs.ADCRESULT3;
678
679     // Here covert ADCIND0 to volts
680     adcC2Volt = adcc2result*3.0/4095.0;
681     adcC3Volt = adcc3result*3.0/4095.0;
682     adcC4Volt = adcc4result*3.0/4095.0;

```



```

683     adcC5Volt = adcc5result*3.0/4095.0;
684
685     if (MPU9250ignoreCNT >= 1) {
686         CurrentChip = MPU9250;
687         SpiRegs.SPIFFCT.bit.TXDLY = 0;
688         SpiRegs.SPIFFRX.bit.RXFFIL = 8;
689
690         GpioDataRegs.GPCLEAR.bit.GPIO66 = 1;
691
692         SpiRegs.SPITXBUF = ((0x8000)|(0x3A00));
693         SpiRegs.SPITXBUF = 0;
694         SpiRegs.SPITXBUF = 0;
695         SpiRegs.SPITXBUF = 0;
696         SpiRegs.SPITXBUF = 0;
697         SpiRegs.SPITXBUF = 0;
698         SpiRegs.SPITXBUF = 0;
699         SpiRegs.SPITXBUF = 0;
700     } else {
701         MPU9250ignoreCNT++;
702     }
703
704     numADCCcalls++;
705     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
706     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
707     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
708
709 }
710
711 // cpu_timer0_isr - CPU Timer0 ISR
712 __interrupt void cpu_timer0_isr(void)
713 {
714     CpuTimer0.InterruptCount++;
715
716     numTimer0calls++;
717
718     if ((numTimer0calls%5) == 0) {
719         // Blink LaunchPad Red LED
720         //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
721     }
722
723     // Acknowledge this interrupt to receive more interrupts from group 1
724     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
725 }
726
727 // cpu_timer1_isr - CPU Timer1 ISR
728 __interrupt void cpu_timer1_isr(void)
729 {
730     serial_sendSCIC(&SerialC, G_command, G_len);
731
732     CpuTimer1.InterruptCount++;
733 }
734
735 // cpu_timer2_isr CPU Timer2 ISR
736 __interrupt void cpu_timer2_isr(void)
737 {
738     // Blink LaunchPad Blue LED
739     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
740
741     CpuTimer2.InterruptCount++;
742
743     PostSWI3();
744
745     // if ((CpuTimer2.InterruptCount % 10) == 0) {
746     //     UARTPrint = 1;
747     // }
748 }
749
750 void setF28027EPWM1A(float controleffort){
751     if (conroleffort < -10) {
752         controleffort = -10;
753     }
754     if (conroleffort > 10) {
755         controleffort = 10;
756     }
757     float value = (conroleffort+10)*3000.0/20.0;
758     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
759 }
760
761 void setF28027EPWM2A(float controleffort){
762     if (conroleffort < -10) {
763         controleffort = -10;
764     }
765     if (conroleffort > 10) {
766         controleffort = 10;
767     }
768     float value = (conroleffort+10)*3000.0/20.0;
769     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
770 }
771
772 //
773 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
774 //
775 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
776 {
777     GpioDataRegs.GPASET.bit.GPIO61 = 1;
778     // Set interrupt priority:
779     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
780     IER |= M_INT12;
781     IER &= MINT12; // Set "global" priority
782     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
783     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
784     __asm(" NOP");
785     EINT;
786
787     //#####
788     IMU_data[0] = readdata[1];
789     IMU_data[1] = readdata[2];
790     IMU_data[2] = readdata[3];
791     IMU_data[3] = readdata[5];
792     IMU_data[4] = readdata[6];
793     IMU_data[5] = readdata[7];
794
795     accelx = (((float)(IMU_data[0]))*4.0/32767.0);

```

```

797 accelx = (((float)(IMU_data[1]))*4.0/32767.0);
798 accelz = (((float)(IMU_data[2]))*4.0/32767.0);
799 gyrox = (((float)(IMU_data[3]))*250.0/32767.0);
800 gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
801 gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
802
803 if(calibration_state == 0){
804     calibration_count++;
805     if (calibration_count == 2000) {
806         calibration_state = 1;
807         calibration_count = 0;
808     }
809 } else if(calibration_state == 1){
810     accelx_offset+=accelx;
811     accely_offset+=accely;
812     accelz_offset+=accelz;
813     gyrox_offset+=gyrox;
814     gyroy_offset+=gyroy;
815     gyroz_offset+=gyroz;
816     gyroLPR510_offset+=adcC5Volt;
817     calibration_count++;
818     if (calibration_count == 2000) {
819         calibration_state = 2;
820         accelx_offset/=2000.0;
821         accely_offset/=2000.0;
822         accelz_offset/=2000.0;
823         gyrox_offset/=2000.0;
824         gyroy_offset/=2000.0;
825         gyroz_offset/=2000.0;
826         gyroLPR510_offset/=2000.0;
827         calibration_count = 0;
828     }
829 } else if(calibration_state == 2) {
830
831     gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
832     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
833
834     gyro9250_drift += (((gyroz-gyro_offset) + old_gyro9250)*.0005)/(2000);
835     old_gyro9250 = gyroz-gyro_offset;
836     calibration_count++;
837
838     if (calibration_count == 2000) {
839         calibration_state = 3;
840         calibration_count = 0;
841         doneCal = 1;
842         newOPTITRACKpose = 0;
843     }
844 } else if(calibration_state == 3){
845     accelx -=(accelx_offset);
846     accely -=(accely_offset);
847     accelz -=(accelz_offset);
848     gyrox -= gyro_offset;
849     gyroy -= gyroy_offset;
850     gyroz -= gyro_offset;
851     LeftWheel = readEncLeft();
852     RightWheel = readEncRight();
853     HandValue = readEncWheel();
854
855     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
856     old_gyro9250 = gyroz;
857
858     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
859     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
860
861     gyro9250_radians = (gyro9250_angle * (PI/180.0));
862     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
863
864     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
865     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
866
867     if (newLinuxCommands == 1) {
868         newLinuxCommands = 0;
869         printLinux1 = LinuxCommands[0];
870         printLinux2 = LinuxCommands[1];
871         /// = LinuxCommands[2];
872         /// = LinuxCommands[3];
873         /// = LinuxCommands[4];
874         /// = LinuxCommands[5];
875         /// = LinuxCommands[6];
876         /// = LinuxCommands[7];
877         /// = LinuxCommands[8];
878         /// = LinuxCommands[9];
879         /// = LinuxCommands[10];
880     }
881
882     if (NewCAMDataThreshold1 == 1) {
883         NewCAMDataThreshold1 = 0;
884         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
885         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
886         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
887
888         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
889         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
890         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
891
892         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
893         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
894         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
895         numThres1++;
896         if ((numThres1 % 5) == 0) {
897             // LED4 is GPIO97
898             GpioDataRegs.GPDT0GGLE.bit.GPIO97 = 1;
899         }
900     }
901
902     if (NewCAMDataThreshold2 == 1) {
903         NewCAMDataThreshold2 = 0;
904         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
905         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
906         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
907
908         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
909         NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
910         NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];

```



```

911     NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
912     NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
913     NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
914     numThres2++;
915     if ((numThres2 % 5) == 0) {
916         // LED5 is GPIO111
917         GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
918     }
919 }
920
921
922 if (NewCAMDataAprilTag1 == 1) {
923     NewCAMDataAprilTag1 = 0;
924     tagx = fromCAMvaluesAprilTag1[0];
925     tagy = fromCAMvaluesAprilTag1[1];
926     tagz = fromCAMvaluesAprilTag1[2];
927
928     tagthetax = fromCAMvaluesAprilTag1[3];
929     tagthetay = fromCAMvaluesAprilTag1[4];
930     tagthetaz = fromCAMvaluesAprilTag1[5];
931
932     tagid = fromCAMvaluesAprilTag1[6];
933
934     numtag++;
935     if ((numtag % 5) == 0) {
936         // LED2 is GPIO94
937         GpioDataRegs.GPCTOGGLE.bit.GPIO94 = 1;
938     }
939 }
940
941 if (NewLVData == 1) {
942     NewLVData = 0;
943     printLV1 = fromLVvalues[0];
944     printLV2 = fromLVvalues[1];
945     //?? = fromLVvalues[2];
946     //?? = fromLVvalues[3];
947     //?? = fromLVvalues[4];
948     //?? = fromLVvalues[5];
949     //?? = fromLVvalues[6];
950     //?? = fromLVvalues[7];
951 }
952
953
954 if (new_optitrack == 1) {
955     OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
956     new_optitrack = 0;
957 }
958
959 // Step 0: update B, u
960 B[0][0] = cosf(ROBOTps.theta)*0.001;
961 B[1][0] = sinf(ROBOTps.theta)*0.001;
962 B[2][1] = 0.001;
963 u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
964 u[1][0] = gyro*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
965
966 // Step 1: predict the state and estimate covariance
967 Matrix3x2_Mult(B, u, Bu); // Bu = B*u
968 Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
969 Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
970 // Step 2: if there is a new measurement, then update the state
971 if (1 == newOPTITRACKpose) {
972     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
973     if ((OptiNumUpdatesProcessed%10)==0) {
974         // LED 3
975         GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
976     }
977     newOPTITRACKpose = 0;
978     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
979     z[1][0] = OPTITRACKps.y;
980     // fix for OptiTrack problem at 180 degrees
981     if (cosf(ROBOTps.theta) < -0.99) {
982         z[2][0] = ROBOTps.theta;
983     }
984     else {
985         z[2][0] = OPTITRACKps.theta;
986     }
987     // Step 2a: calculate the innovation/measurement residual, ytilde
988     Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
989     // Step 2b: calculate innovation covariance, S
990     Matrix3x3_Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
991     // Step 2c: calculate the optimal Kalman gain, K
992     Matrix3x3_Invert(S, S_inv);
993     Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
994     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
995     Matrix3x1_Mult(K, ytilde, temp_3x1);
996     Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
997     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
998     Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
999     Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1000 } // end of correction step
1001
1002 // set ROBOTps to the updated and corrected Kalman values.
1003 ROBOTps.x = x_pred[0][0];
1004 ROBOTps.y = x_pred[1][0];
1005 ROBOTps.theta = x_pred[2][0];
1006
1007 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1008 // uses xy code to step through an array of positions
1009 if (xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1010     statePos = (statePos+1)%NUMWAYPOINTS;
1011 }
1012 // state machine
1013 switch (RobotState) {
1014 case 1:
1015
1016     // vref and turn are the vref and turn returned from xy_control
1017
1018     if (LADARfront < 1.2) {
1019         vref = 0.2;
1020         checkfronttally++;
1021         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1022             RobotState = 10; // Wall follow
1023             WallFollowtime = 0;
1024             right_wall_follow_state = 1;

```

```

1025     }
1026 } else {
1027     checkfronttally = 0;
1028 }
1029
1030 break;
1031 case 10:
1032     if (right_wall_follow_state == 1) {
1033         //Left Turn
1034         turn = Kp_front_wall*(14.5 - LADARfront);
1035         vref = front_turn_velocity;
1036         if (LADARfront > left_turn_Stop_threshold) {
1037             right_wall_follow_state = 2;
1038         }
1039     } else if (right_wall_follow_state == 2) {
1040         //Right Wall Follow
1041         turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
1042         vref = foward velocity;
1043         if (LADARfront < left_turn_Start_threshold) {
1044             right_wall_follow_state = 1;
1045         }
1046     }
1047     if (turn > turn_saturation) {
1048         turn = turn_saturation;
1049     }
1050     if (turn < -turn_saturation) {
1051         turn = -turn_saturation;
1052     }
1053
1054     WallFollowtime++;
1055     if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1056         RobotState = 1;
1057         checkfronttally = 0;
1058     }
1059     break;
1060
1061 case 20:
1062     // put vision code here
1063     break;
1064 default:
1065     break;
1066 }
1067
1068
1069 //Must be called each time into this SWI1 function
1070 PIconrol(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1071 // These below lines also must be called each time into this SWI1 function
1072 setEPWM1A(uLeft);
1073 setEPWM2A(-uRight);
1074 setF28027EPWM1A(uLeft);
1075 setF28027EPWM2A(-uRight);
1076 LeftWheel_1 = LeftWheel;
1077 RightWheel_1 = RightWheel;
1078
1079 if((timecount%250) == 0) {
1080     DataToLabView.floatData[0] = ROBOTps.x;
1081     DataToLabView.floatData[1] = ROBOTps.y;
1082     DataToLabView.floatData[2] = ROBOTps.theta;
1083     DataToLabView.floatData[3] = (float)timecount;
1084     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1085     DataToLabView.floatData[5] = (float)RobotState;
1086     DataToLabView.floatData[6] = (float)statePos;
1087     DataToLabView.floatData[7] = LADARfront;
1088     LVsenddata[0] = '*'; // header for LVdata
1089     LVsenddata[1] = '$';
1090     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1091         if (i%2==0) {
1092             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1093         } else {
1094             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1095         }
1096     }
1097     serial_sendSCID(&SerialID, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1098 }
1099
1100 }
1101 timecount++;
1102 if((timecount%200) == 0)
1103 {
1104     if(doneCal == 0) {
1105         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1106     }
1107     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1108     UARTPrint = 1; // Tell While loop to print
1109 }
1110
1111 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1112 CurrentChip = DAN28027;
1113 SpibRegs.SPIFFRX.bit.RXFFIL = 3;
1114 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1115 SpibRegs.SPITXBUF = 0x00DA;
1116 SpibRegs.SPITXBUF = EPwm1A_F28027;
1117 SpibRegs.SPITXBUF = EPwm2A_F28027;
1118
1119
1120 //#####
1121 //
1122 // Restore registers saved:
1123 //
1124 DINT;
1125 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1126
1127 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1128 }
1129
1130 //
1131 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1132 //
1133 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1134 {
1135     // Set interrupt priority:
1136     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1137     IER |= M_INT12;
1138     IER   &= MINT12; // Set "global" priority

```

```

1139 PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1140 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1141 __asm(" NOP");
1142 EINT;
1143
1144 //#####
1145 // Insert SWI ISR Code here.....
1146 // LED1
1147 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1148 if (LADArpingpong == 1) {
1149     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1150     LADARrightfront = 19; // 19 is greater than max feet
1151     for (LADARi = 52; LADARi <= 56; LADARi++) {
1152         if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1153             LADARrightfront = ladar_data[LADARi].distance_ping;
1154         }
1155     }
1156     // LADARfront is the min of dist 111, 112, 113, 114, 115
1157     LADARfront = 19;
1158     for (LADARi = 111; LADARi <= 115; LADARi++) {
1159         if (ladar_data[LADARi].distance_ping < LADARfront) {
1160             LADARfront = ladar_data[LADARi].distance_ping;
1161         }
1162     }
1163     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta));
1164     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)+LADARps.y*cosf(ROBOTps.theta));
1165     for (LADARi = 0; LADARi < 228; LADARi++) {
1166
1167         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1168         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1169
1170     }
1171 } else if (LADArpingpong == 0) {
1172     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1173     LADARrightfront = 19; // 19 is greater than max feet
1174     for (LADARi = 52; LADARi <= 56; LADARi++) {
1175         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1176             LADARrightfront = ladar_data[LADARi].distance_pong;
1177         }
1178     }
1179     // LADARfront is the min of dist 111, 112, 113, 114, 115
1180     LADARfront = 19;
1181     for (LADARi = 111; LADARi <= 115; LADARi++) {
1182         if (ladar_data[LADARi].distance_pong < LADARfront) {
1183             LADARfront = ladar_data[LADARi].distance_pong;
1184         }
1185     }
1186     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1187     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1188     for (LADARi = 0; LADARi < 228; LADARi++) {
1189
1190         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1191         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1192
1193     }
1194 }
1195
1196
1197
1198
1199 //#####
1200 //
1201 // Restore registers saved:
1202 //
1203 DINT;
1204 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1205
1206 }
1207
1208 //
1209 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1210 //
1211 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1212 {
1213     // Set interrupt priority:
1214     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1215     IER |= M_INT12;
1216     IER &= MINT12; // Set "global" priority
1217     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1218     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1219     __asm(" NOP");
1220     EINT;
1221
1222     //#####
1223     // Insert SWI ISR Code here.....
1224
1225     RCangle = readEncWheel();
1226     setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1227     setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1228     setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90
1229     setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90
1230     setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90
1231
1232     //#####
1233     //
1234     // Restore registers saved:
1235     //
1236     DINT;
1237     PieCtrlRegs.PIEIER12.all = TempPIEIER;
1238
1239 }
1240
1241 // ----- code for CAN start here -----
1242 __interrupt void can_isr(void)
1243 {
1244     int i = 0;
1245
1246     uint32_t status;
1247
1248     //
1249     // Read the CAN interrupt status to find the cause of the interrupt
1250     //
1251     status = CANGetInterruptCause(CANB_BASE);
1252

```

```

1253 //
1254 // If the cause is a controller status interrupt, then get the status
1255 //
1256 if(status == CAN_INT_INT0ID_STATUS)
1257 {
1258     //
1259     // Read the controller status. This will return a field of status
1260     // error bits that can indicate various errors. Error processing
1261     // is not done in this example for simplicity. Refer to the
1262     // API documentation for details about the error status bits.
1263     // The act of reading this status will clear the interrupt.
1264     //
1265     status = CANgetStatus(CANB_BASE);
1266 }
1267
1268 //
1269 // Check if the cause is the receive message object 2
1270 //
1271 //
1272 else if(status == RX_MSG_OBJ_ID_1)
1273 {
1274     //
1275     // Get the received message
1276     //
1277     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1278
1279     for(i = 0; i<2; i++)
1280     {
1281         dis_raw_1[i] = rxMsgData[i];
1282     }
1283
1284     dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1285
1286     measure_status_1 = rxMsgData[2];
1287
1288     //
1289     // Getting to this point means that the RX interrupt occurred on
1290     // message object 2, and the message RX is complete. Clear the
1291     // message object interrupt.
1292     //
1293     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1294
1295     //
1296     // Increment a counter to keep track of how many messages have been
1297     // received. In a real application this could be used to set flags to
1298     // indicate when a message is received.
1299     //
1300     rxMsgCount_1++;
1301
1302     //
1303     // Since the message was received, clear any error flags.
1304     //
1305     errorFlag = 0;
1306 }
1307
1308 else if(status == RX_MSG_OBJ_ID_2)
1309 {
1310     //
1311     // Get the received message
1312     //
1313     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1314
1315     for(i = 0; i<2; i++)
1316     {
1317         dis_raw_2[i] = rxMsgData[i];
1318     }
1319
1320     dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1321
1322     measure_status_2 = rxMsgData[2];
1323
1324     //
1325     // Getting to this point means that the RX interrupt occurred on
1326     // message object 2, and the message RX is complete. Clear the
1327     // message object interrupt.
1328     //
1329     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1330
1331     //
1332     // Increment a counter to keep track of how many messages have been
1333     // received. In a real application this could be used to set flags to
1334     // indicate when a message is received.
1335     //
1336     rxMsgCount_2++;
1337
1338     //
1339     // Since the message was received, clear any error flags.
1340     //
1341     errorFlag = 0;
1342 }
1343
1344 else if(status == RX_MSG_OBJ_ID_3)
1345 {
1346     //
1347     // Get the received message
1348     //
1349     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1350
1351     for(i = 0; i<4; i++)
1352     {
1353         lightlevel_raw_1[i] = rxMsgData[i];
1354         quality_raw_1[i] = rxMsgData[i+4];
1355     }
1356
1357     lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1358     quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1359
1360     //
1361     // Getting to this point means that the RX interrupt occurred on
1362     // message object 2, and the message RX is complete. Clear the
1363     // message object interrupt.
1364     //
1365     //
1366     //

```

```

1367     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1368
1369     //
1370     // Since the message was received, clear any error flags.
1371     //
1372     errorFlag = 0;
1373 }
1374
1375
1376 else if(status == RX_MSG_OBJ_ID_4)
1377 {
1378     //
1379     // Get the received message
1380     //
1381     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1382
1383     for(i = 0; i<4; i++)
1384     {
1385         lightlevel_raw_2[i] = rxMsgData[i];
1386         quality_raw_2[i] = rxMsgData[i+4];
1387     }
1388
1389     lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1390     quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1391
1392     //
1393     // Getting to this point means that the RX interrupt occurred on
1394     // message object 2, and the message RX is complete. Clear the
1395     // message object interrupt.
1396     //
1397     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1398
1399     //
1400     // Since the message was received, clear any error flags.
1401     //
1402     errorFlag = 0;
1403 }
1404
1405
1406
1407
1408 //
1409 // If something unexpected caused the interrupt, this would handle it.
1410 //
1411 else
1412 {
1413     //
1414     // Spurious interrupt handling can go here.
1415     //
1416 }
1417
1418 //
1419 // Clear the global interrupt flag for the CAN interrupt line
1420 //
1421 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1422
1423 //
1424 // Acknowledge this interrupt located in group 9
1425 //
1426 InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1427 }
1428 // ----- code for CAN end here -----

```